Portaldot Hacker House 2026 · Ecosystem DApps

Reachpot

Get paid for your reach, fairly.

Pro-rata creator campaigns on Portaldot V3, funded in any Portaldot native asset.

reachpot.vercel.app | github.com/lazydevpro/reachpot | Live on Portaldot V3 testnet

The problem

Paying creators per view is broken.

Brands already pay "clippers" to post Shorts about them. Fixed-CPM programs fail creators in three ways:

First-come drain

Budgets run out early. Late creators with great posts earn nothing.

Opaque payouts

The platform decides what counts and pays off-chain, when and where it likes.

Rejection pays

Rejecting a submission saves the brand money, so creators carry all the risk.

And on Portaldot: new projects need distribution and real utility for their tokens, and creators need a way to turn a project token into something they can use.

The solution

A pot, escrowed on-chain, split by views.

A brand locks a pot in any Portaldot token. Creators post Shorts. When the window closes, the pot splits pro-rata by views, with a per-creator cap, and every creator is paid on-chain.

100%

of the pot is spent. A capped creator's excess re-splits across everyone else ("water-filling").

0

ways for the brand to claw back a live pot. Only the never-allocated remainder is refundable.

1 tap

for a creator to cash a project-token payout out to POT on Portaldot's native DEX.

How it works

From pot to payout, verifiable at every step.

1 · FUND

Brand locks a pot

approve via the native-asset ERC-20 precompile → `createCampaign`; rules hash pinned on-chain

2 · SUBMIT

Creators post Shorts

Portaldot wallet signs each submission; payouts bound to the creator's account

3 · SNAPSHOT

Keeper splits by views

snapshot stored in TransactionStorage; Merkle root + blake2 hash posted to the contract

4 · DISPUTE

Anyone verifies

re-read the snapshot from chain, recompute the root in the browser; guardian can cancel a bad root

5 · PAY

Everyone is paid

`payoutMany` pushes payouts; a failing recipient never blocks the batch

6 · OWN

Badge + cash-out

soulbound creator badge (Nfts) levels up; swap to POT on AssetConversion

Keeper: snapshot → root → payouts → badges, ticked every minute

Trust: custody & limits trustless · split verifiable · views by oracle (v1)

Why Portaldot is essential

Native modules and EVM, working as one product.

Revive EVM + native-asset ERC-20 precompiles	Reachpot.sol escrows native **Assets** tokens directly — any project token works, no wrapping, no bridge.
H160 ↔ native account mapping	Brands fund from MetaMask; creators use a Portaldot wallet; after `revive.mapAccount`, EVM payouts land in their native account.
AssetConversion (native DEX)	Creators swap payouts to POT in one tap with live on-chain quotes; campaigns add volume to the token's pool.
TransactionStorage	The allocation snapshot is published on-chain; the contract pins its hash and location so anyone can re-derive the root.
Nfts	Soulbound creator badges whose attributes (campaigns, views, earned) are a portable on-chain résumé.

On a plain EVM chain this needs a separate DEX, NFT contract and data-availability layer.

Proof it works

Live on Portaldot V3 testnet, end to end.

Contract `0xcfcdb8cc…d493cf` · deployed at block 150,033

Snapshot stored in TransactionStorage at block **157,841**

2 creators paid 4,000 rUSD each (capped); brand refunded 2,000

Soulbound badges minted; cash-out on **AssetConversion**

Independent check: on-chain bytes → blake2 hash → recomputed root ✔

23 + 17

contract tests (unit, fuzz, golden fixtures shared with TS) + allocation tests · CI on every push

Live

reachpot.vercel.app — Next.js app + Postgres + keeper, auto-deployed from GitHub

Every transaction hash: docs/e2e-testnet.md

Deployment: deployments/testnet.json

Engineered for Portaldot

Designed around Revive's storage deposits.

On Portaldot, deploying costs ~0.078 POT per byte of initcode (0.06/byte code deposit + 30% code-hash lockup). We measured it, then built for it.

7.9 → 5.9 KB

Contract slimmed (no admin surface, shared transfer helper, size-optimised build): deploy **615 → 448** tPOT.

~19 · ~16

tPOT deposits per campaign and per payout root; ~4 per creator paid; ~3.3 to map a creator account.

Documented

Costs, the precompile address layout and the H160 mapping flow are written up for other builders.

Business model & growth

A growth engine for Portaldot projects.

Who pays

Portaldot projects launching tokens, and Web2 brands running creator campaigns. They fund pots; creators earn.

Revenue (planned)

A small protocol fee on each pot, verified-analytics tiers for brands, and featured campaign placement.

The loop

A project funds a pot in its own token → creators post and get paid → creators cash out on the native DEX → the token gains liquidity and holders, and Portaldot gains users who arrived through a Short.

Why it wins

Fair by construction (pro-rata, capped, no claw-back), verifiable by anyone, and paid in seconds instead of net-30.

Roadmap & why support it

From hackathon to Portaldot mainnet.

Next (Nov)

Real YouTube verification live (Data API + channel binding code), guardian UI, sponsored creator onboarding (MetaTx / mapping deposit).

Mainnet

Launch with Portaldot ecosystem projects as the first brands; audits; protocol fee switch.

Later

TikTok, X and Instagram Reels; attested view data (OAuth / zkTLS) to shrink oracle trust; brands from other chains.

The ask: mainnet launch support, co-marketing with Portaldot projects as launch brands, and deposit/gas subsidies so first-time creators onboard free.



Make every view count.

reachpot.vercel.app

github.com/lazydevpro/reachpot

Reachpot · Portaldot Hacker House 2026 · built solo by lazydevpro